

APP.PY

1. Første så skrevet jeg planen om hva prosjektet mitt skulle være om.
2. Etter hvert gå jeg til terminalen i min egen PC og oppretter databasen:

```
stefanosgkiokas@stefanosgkiokas:~$ mariadb -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 11.4.7-MariaDB-0ubuntu0.25.04.1 Ubuntu 25.04

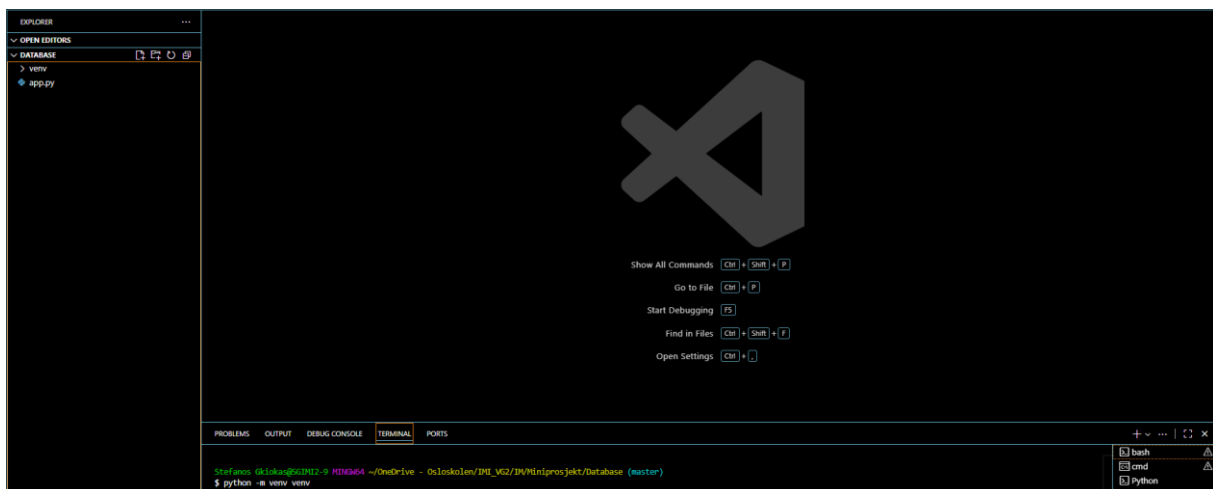
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Support MariaDB developers by giving a star at https://github.com/MariaDB/server
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE vær_kuben_db;
Query OK, 1 row affected (0.002 sec)
```

Feilsøk: ikke bruke spesielle bokstaver som står på bildet. SQL er veldig presis, og SQL har ikke en kode for å endre navnet til en database.

3. Nå lager man en mappe for din venv. mappe og din app.py:



- Åpne en «bash» terminal
- Skriv «python -m venv venv»
- Utafor venv. mappen, lag en py fil.

4. Etter du her opprettet app.py og venv:

- HUSK: Pass på at du har lastet ned «flask», «mysql-connector-python» og «requests»

- Hvis ikke: skriv inn «bash» terminal: «pip install flask» og «pip install mysql.connector» og «pip install requests»
- For å dobbelsjekk: «pip list»

```
$ pip list
-----
bcrypt                5.0.0
blinker                1.9.0
certifi                2025.11.12
charset-normalizer     3.4.4
click                  8.3.0
colorama               0.4.6
Flask                  3.1.2
idna                   3.11
itsdangerous           2.2.0
Jinja2                 3.1.6
MarkupSafe             3.0.3
mysql-connector-python 9.4.0
packaging              25.0
pip                    25.3
requests               2.32.5
urllib3                2.5.0
Werkzeug               3.1.3
```

5. Vi må opprette en «table» etter vi er koblet databasen med ip-adressen til linux og brukernavn, passord til maria_db brukeren i py filen eller vi kan opprette det via MariaDB(som står i bildet):

- Man kan lage bruke «CREATE TABLE navn;»

```
Database changed
MariaDB [vaer_kuben_db]> SHOW TABLES;
+-----+
| Tables_in_vaer_kuben_db |
+-----+
| vaerdata                  |
+-----+
1 row in set (0.001 sec)
```

6. For å vise den nøyaktig været så trenger man den riktig tid og lokasjon.

- Her så bruker man datetime-modul:
- «dataclass» er en hjelpemodul i Python som brukes til å lage enkle data-objekter.

```
1 import requests
2 import datetime
3 import mysql.connector
4 from flask import Flask, render_template, g
5 from dataclasses import dataclass
6
7 app = Flask(__name__)
8
```

7. Når må vi skrive dataclassen, her bruker vi «WeatherData» og den beskrive hvordan ett vær-objekt skal se ut.

```
12
13 @dataclass
14 class WeatherData:
15     lokasjon: str
16     temperatur: float
17     beskrivelse: str
18     sist_oppdateret: datetime.datetime | None = None
19
20 #
```

8. Sjekker om tilkoblingen allerede finnes i Flask sin globale lagring (g) og etablerer ny tilkobling hvis den mangler.

```
23
24 DB_CONFIG = {
25     "host": "10.200.14.18",
26     "user": "stefanosgkiokas",
27     "password": "IMIKuben1337!",
28     "database": "vaer_kuben_db"
29 }
30
31 CACHE_DURATION = datetime.timedelta(minutes=60)
32 LOKASJON = "Kuben VGS, Oslo"
33
34 # =====
35 # DATABASE HÅNDTERING
36 # =====
37
38 def get_db():
39     if 'db' not in g:
40         g.db = mysql.connector.connect(**DB_CONFIG)
41     return g.db
42
```

9. Vi lukker databasetilkoblingen etter at hver brukerforespørsel er ferdigbehandlet. Dette forhindrer at vi får for mange åpne tilkoblinger som tetter serveren.

```
24
25 @app.teardown_appcontext
26 def close_db(error):
27     db = g.pop('db', None)
28     if db is not None and db.is_connected():
29         db.close()
30
```

10. Oppretter 'vaerdata' tabellen og denne funksjonen kjøres kun én gang ved oppstart av applikasjonen.

```
49
50 def create_table_if_not_exists():
51     SQL_CREATE_TABLE = """
52     CREATE TABLE IF NOT EXISTS vaerdata (
53         id INT PRIMARY KEY,
54         lokasjon VARCHAR(255) NOT NULL,
55         temperatur DECIMAL(4,1),
56         beskrivelse VARCHAR(255),
57         sist_oppdattert DATETIME NOT NULL
58     );
59     """
60
61     try:
62         conn = mysql.connector.connect(**DB_CONFIG)
63         cursor = conn.cursor()
64         cursor.execute(SQL_CREATE_TABLE)
65         conn.commit()
66         print("Tabellen 'vaerdata' er klar.")
67     except mysql.connector.Error as err:
68         print(f"FEIL ved DB-oppsett: {err}")
69     finally:
70         if conn.is_connected():
71             conn.close()
```

Feilhåndtering: Det hjelper å se om det er en feil med passordet/brukeren og det sikrer at tilkoblingene lukes, uansett om det er feil eller suksess.

```
53
54 except mysql.connector.Error as err:
55     print(f"FEIL ved DB-oppsett: {err}. Sjekk passord/brukernavn.")
56 finally:
57     if 'conn' in locals() and conn.is_connected():
58         conn.close()
59
60
```

11. Hente ekte værdata fra Met.no API. Et API (som står for "Application Programming Interface") er rett og slett en servitør for programvare.

- Vi trenger koordinatar for Kuben VGS/Økern i Oslo
- Og Met.no krever en User-Agent for å identifisere app-en

```
def fetch_weather_from_api(location_name):
    lat, lon = 59.939, 10.806
    URL = f"https://api.met.no/weatherapi/locationforecast/2.0/compact?lat={lat}&lon={lon}"
    headers = {'User-Agent': 'KubenVærProsjekt/1.0 studentnavn@skole.no'}
```

12. For å sjekke om statuskoden er OK. Kaster feil ellers. Og for å gjøre JSON-data om til Python dictionary.

```
70     try:
71         response = requests.get(URL, headers=headers)
72         response.raise_for_status()
73         data = response.json()
74
```

13. For å bruke datastrukturer for å finne relevant data (været for nærmest time)

```
try:
    response = requests.get(URL, headers=headers)
    response.raise_for_status()
    data = response.json()
    timeseries = data['properties']['timeseries']
    neste_time = timeseries[0]['data']
    temperatur = neste_time['instant']['details']['air_temperature']
    symbol = neste_time['next_1_hours']['summary']['symbol_code'].replace('_', ' ').title()
```

14. For å returnere et dictionary med de hentede dataene

```
78
79     return {
80         'temperatur': temperatur,
81         'beskrivelse': symbol,
82         'lokasjon': location_name
83     }
84
85 except requests.exceptions.RequestException as e:
86     print(f"FEIL ved API-kall til Met.no: {e}")
87     return None
88 except Exception as e:
89     print(f"FEIL ved parsing av JSON fra Met.no: {e}")
90     return None
91
```

15. Vi trenger koder for å lagre eller oppdaterer værdata i databasen.

- Med en kommando at hvis ID 1 finnes oppdater hvis ikke opprett det.

```
92 def update_db_with_weather(data):
93     db = get_db()
94     cursor = db.cursor()
95     sql_insert_update = """
96     INSERT INTO vaerdata (id, lokasjon, temperatur, beskrivelse, sist_oppdatert)
97     VALUES (1, %s, %s, %s, NOW())
98     ON DUPLICATE KEY UPDATE
99         lokasjon = VALUES(lokasjon),
100         temperatur = VALUES(temperatur),
101         beskrivelse = VALUES(beskrivelse),
102         sist_oppdatert = VALUES(sist_oppdatert);
103     """
104     cursor.execute(sql_insert_update, (data['lokasjon'], data['temperatur'], data['beskrivelse']))
105     db.commit()
```

16. For å prøve å hente de legrede data og å hente den ene raden:

```
111 @app.route('/')
112 def vaer_dashboard():
113     db = get_db()
114     cursor = db.cursor(dictionary=True)
115     cursor.execute("SELECT * FROM vaerdata WHERE id = 1")
116     vaer_data = cursor.fetchone()
117     cache_status = "Databasen er tom."
118     display_data = None
```

- dictionary=True gjør at resultatet blir en Python dict, enklere å jobbe med

17. Dobbelsjekke om det ER data i databasen, regne ut hvor lang tid det er siden sist oppdatering:

```
119
120     if vaer_data:
121         tid_siden_oppdatering = datetime.datetime.now() - vaer_data['sist_oppdatert']
122
123         if tid_siden_oppdatering < CACHE_DURATION:
124             display_data = vaer_data
125             minutter_gammel = int(tid_siden_oppdatering.total_seconds() / 60)
126             cache_status = f"Ferske data fra MariaDB. Oppdatert for {minutter_gammel} minutter siden."
127
128             return render_template('index.html', vaer=display_data, cache_status=cache_status)
129
```

- For å sjekke om dataene er ferske nok ved «Cache»(CACHE HIT) og returnere HTML-malen med de raske datene.

18. Hvis «CACHE MISS», data er gamle eller databasen er tom da kommer vi til å bruke noen hvis og eller- funksjoner.

```
130     nytt_vaer = fetch_weather_from_api(LOKASJON)
131     if nytt_vaer:
132         # Hvis API-kallet lyktes:
133         update_db_with_weather(nytt_vaer)
134         display_data = nytt_vaer
135         display_data['sist_oppdatert'] = datetime.datetime.now()
136         cache_status = "Data hentet fra Met.no API og lagret i MariaDB."
137     else:
138         # Hvis API-kallet feilet:
139         display_data = vaer_data
140         cache_status = "FEIL: Klarte ikke hente nytt vær. Viser gamle data hvis tilgjengelig."
141
142     return render_template('index.html', vaer=display_data, cache_status=cache_status)
143
```

- Hvis API-kallet lyktes:
 - Lagrer de nye dataene i MariaDB
 - Setter nytt tidsstempel
- Eller hvis API-kallet feilet:

- Viser gamle data som en fallback
- Returnerer HTML-malen med de nyeste dataene

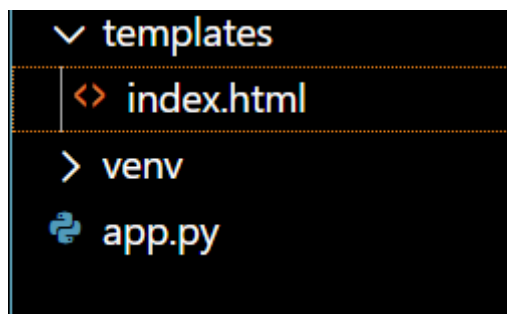
19. Og siste delen:

- Så trenger man kodene til at app.py skal kjøres direkte
- Og kodene for starter Flask-serveren, tilgjengelig for andre på nettverket (0.0.0.0). I filen app.py må det stå:
«app.run(host='0.0.0.0', port=5000)»
- **0.0.0.0** er en spesiell kode som betyr: "Lytt på ALLE IP-adresser denne maskinen har".

```
144
145  ✓ if __name__ == '__main__':
146      create_table_if_not_exists()
147      app.run(host='0.0.0.0', port=5000, debug=True)
```

Template (HTML)

1. Når trenger vi en template. En template er en HTML-fil som Flask fyller med data før den sendes til nettleseren.
2. Vi skal lage en mappe vi kaller det «templates» og vi oppretter en HTML-fil innafor der.



HUSK: At mappestruktur er VELDIG viktig her. Din «app.py» burde IKKE være inni samme mappe som din HTML-fil.

3. Det skriver inn «!» og får du den her:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>

</body>
</html>
```

- Nå må vi endre litt på ting innafor «head»:
- «title» skriv inn «Kuben Værvarsel»
- Så vil jeg bruke «Tailwind CSS» i standen for den vanlig CSS for å gjør det lettere for meg selv. Tailwind CSS er et CSS-rammeverk som lar deg style nettsider direkte i HTML ved hjelp av ferdige klasser.

```
1 <!DOCTYPE html>
2 <html lang="no">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Kuben Værvarsel</title>
7   <script src="https://cdn.tailwindcss.com"></script>
8   <style>
9     @import url('https://fonts.googleapis.com/css2?family=Inter:wght@100..900&display=swap');
10    body {
11      font-family: 'Inter', sans-serif;
12      background-color: #f1f5f9;
13    }
14  </style>
15 </head>
```

4. Etter hvert innafor «body»

- «Body» bruker flexbox for å sentrere innholdet på siden.
- Hovedkortet som viser været
- Skrive en kode for å ha overskrift med lokasjon (kommer fra Flask) og sjekker om værd data finnes.

```
15 </head>
16 <body class="min-h-screen flex items-center justify-center p-4">
17   <div class="bg-white w-full max-w-sm p-6 sm:p-8 rounded-2xl shadow-xl">
18     <h1 class="text-3xl font-extrabold text-gray-800 mb-6 text-center">
19       Været på {{ lokasjon }}
20     </h1>
21
22     {% if vær %}
23
```

5. Når skriver vi koden (Jinja)for å vise hva været og temperatur er:

```
24 | <div>
25 | | <div class="flex items-center justify-center space-x-4 mb-4">
26 | | | <div class="text-5xl">
27 | | | | {% if 'sol' in vær.beskrivelse.lower() or 'klar' in vær.beskrivelse.lower() %}
28 | | | | | 🌞
29 | | | | {% elif 'regn' in vær.beskrivelse.lower() or 'yr' in vær.beskrivelse.lower() %}
30 | | | | | 🌧️
31 | | | | {% elif 'snø' in vær.beskrivelse.lower() %}
32 | | | | | ❄️
33 | | | | {% elif 'sky' in vær.beskrivelse.lower() %}
34 | | | | | ☁️
35 | | | | {% else %}
36 | | | | | 🌫️
37 | | | | {% endif %}
38 | | | </div>
39 | | | <h2 class="text-6xl font-light text-gray-900">
40 | | | | {{ vær.temperatur }}°C
41 | | | </h2>
42 | | </div>
```

6. For å ha info litt mer ned på kortet:

- For å vise bare hvis tidspunkt finnes og for å vise om data kommer fra «cache» eller «API».

```
47 | | <div class="border-t border-gray-200 pt-4 mt-4">
48 | | |
49 | | | {% if vær.sist_oppdatert %}
50 | | | | <p class="text-sm text-gray-500 mb-1">
51 | | | | | Sist oppdatert:
52 | | | | | <span class="font-semibold text-gray-600">
53 | | | | | | {{ vær.sist_oppdatert.strftime('%d.%m %H:%M') }}
54 | | | | | </span>
55 | | | | </p>
56 | | | | {% endif %}
57 | | |
58 | | | <p class="text-sm font-bold text-green-600">
59 | | | | Status: <span>{{ cache_status }}</span>
60 | | | </p>
61 | | </div>
62 | </div>
63 |
```

7. Vi må også skrive koder for hva skjer hvis værdata ikke finnes:

```
64 | | | {% else %}
65 | | | | <div class="text-red-600 bg-red-50 p-4 rounded-lg text-center">
66 | | | | | <p class="font-bold mb-2">Klarte ikke laste værdata.</p>
67 | | | | | <p class="text-sm">Vennligst prøv igjen senere.</p>
68 | | | | </div>
69 | | | | {% endif %}
70 | | |
71 | | </div>
72 |
73 | </body>
74 | </html>
75 |
```

Failsøking



Du kan se på bildet så man kan se and den gamle Python koden funker ikke og jeg fikk ikke noen data fra API-en fra Met.no. Her er hva jeg er endra og hva problemet var:

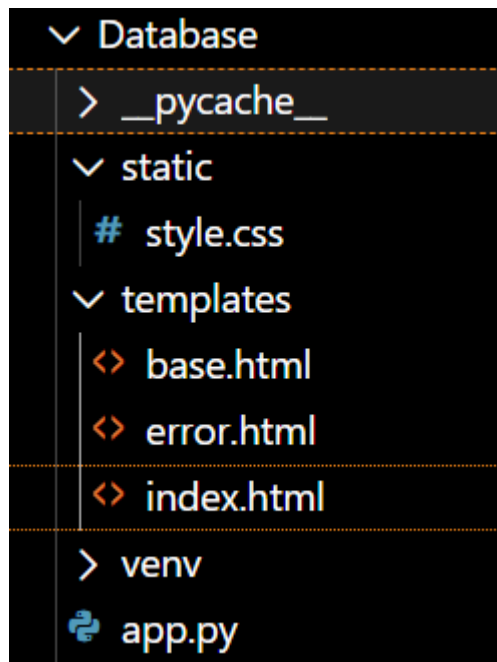
1. Rusberry Pi min kobla ikke til API-en fra Met.no
2. Feil med databasenavn og tabellnavn
3. Manglende tabell i databasen
4. Forvirring rundt Flask templates
5. Problemer med Python-miljø(request)

(Sjekke «readme.md» om hvordan jeg løste disse problemene.)

UKE 10,11,12

Jeg tenke at min Været app burde inneholder mer data om ikke bare kun kuben, men også andre byer i Norge. Så jeg jobbe med mer templates og opptatere app.py-en min. Og opprett en CSS-fil.

Mappestruktur:



Templates (HTML):

- Index.html:

- Templaten ble forbedret ved å strukturere koden med korrekt indentasjon og tydelige seksjoner. I tillegg ble det lagt til funksjonalitet for å velge by dynamisk gjennom et skjema. Koden ble gjort mer lesbar og vedlike holdbar, samtidig som brukeropplevelsen ble forbedret med bedre design og feilhåndtering.

```
1  {% extends "base.html" %}
2
3  {% block title %}
4  Kuben Værvarsel
5  {% endblock %}
6
7  {% block content %}
```

1. Denne templatens arver fra «base.html», og det betyr at du slipper å skrive hele HTML-strukturen på nytt. Og setter tittelen på siden som vises i browser-tab.

```

9   <div class="bg-white w-full max-w-sm p-8 rounded-2xl shadow-xl">
10
11   <h1 class="text-3xl font-bold text-center mb-6">
12   |   Været i {{ lokasjon }}
13   </h1>
14

```

2. Koden på linje 9, lager «kortet» i midten av siden med hvit bakgrunn, maks bredde, padding, rundede hjørner og skygge.
3. Kodene fra linje 11 til 13 er koden for overskrift.
 - «{{ lokasjon }}» kommer fra Python.

```

15   <form method="get" class="text-center mb-6">
16
17   <select name="city" class="border px-3 py-2 rounded">
18
19   |   {% for city in cities %}
20   |   |   <option value="{{ city }}" {% if city == lokasjon %}selected{% endif %}>
21   |   |   |   {{ city }}
22   |   |   </option>
23   |   |   {% endfor %}
24
25   </select>
26   <button type="submit" class="bg-blue-500 text-white px-4 py-2 rounded ml-2 hover:bg-blue-600">
27   |   Vis vær
28   </button>
29 </form>
30
31 {% if vær %}
32

```

4. «form» lager et skjema hvor «get» sender data via URL(f.eks «?city=Oslo»).
5. Koden på linje 16 er for å lage en dropdown meny og jeg navna det «city» så at Flask kan lese dette.
6. Koden i linje 17 er loop-en gjennom liste med byer fra Python.
7. «option» koden i linje 18 lager valg i dropdown og hvirs «city» er nåværende lokasjon den blir valgt.
8. Linje 19 viser navnet på byen.
9. Koden på linje 21 er koden for å slutte loopen.
10. «button» koden i linje 26 er bare knappen som sender skjemaet.

```

31     {% if vær %}
32
33     <div class="text-center">
34         <div class="text-6xl mb-4">
35             {% if "Clear" in vær.beskrivelse %}
36                 🌞
37             {% elif "Rain" in vær.beskrivelse %}
38                 🌧️
39             {% elif "Snow" in vær.beskrivelse %}
40                 ❄️
41             {% elif "Cloud" in vær.beskrivelse %}
42                 ☁️
43             {% else %}
44                 🌫️
45             {% endif %}
46         </div>
47
48         <div class="text-6xl font-light">
49             | {{ vær.temperatur }}°C
50         </div>
51
52         <p class="text-gray-600 mt-3">
53             | {{ vær.beskrivelse }}
54         </p>
55     </div>

```

11. Funksjonen «if» på linje 31 er får hvis værdata finnes det kan returnere info og hvis ikke da viser den error.
12. «div» kodene i linje 34 er for container for vær-ikon. Kodene fra linje 35 til 46 inneholder koder som sjekker værtypen og viser riktig emoji basert på vær (emoji-ene er henta fra ChatGPT).
13. «{{ vær.temperatur }}° C» er koden fra python for å vise temperatur.
14. «{{ if vær.beskrivelse }}

```

55
56     {% if vær.sist_oppdatert %}
57     <p class="text-sm text-gray-500 mt-4">
58         Sist oppdatert:
59         <strong>
60             | {{ vær.sist_oppdatert.strftime('%d.%m %H:%M') }}
61         </strong>
62     </p>
63     {% endif %}

```

15. «{% if vær.sist_oppdatert %}» er koden som sjekker om tidspunkt finnes.
16. «{{ vær.sist_oppdatert.strftime('%d.%m %H:%M') }}

```

64     <p class="text-green-600 text-sm font-bold mt-2">
65         | {{ cache_status }}
66     </p>
67

```

17. «`{{ cache_status }}`» er koden som viser om data er frå cache eller fra API.

```
69     {% else %}
70     <div class="text-center text-red-600">
71     |     Klarte ikke hente værddata
72     </div>
73     {% endif %}
74 </div>
75
76 {% endblock %}
```

18. Eg har også skrevet en feilmelding koden om hvis noe feiler jeg får vite det.
19. «`{% endif %}`» er for å slutte if-setning.
20. «`{% endblock %}`» er for å slutte content-blokken.

Error.html

Feil-templatene ble forbedret ved å strukturere HTML-koden tydeligere og legge til bedre styling. Det ble også gjennomført en knapp som lar brukeren laste siden på nytt. Dette gir en bedre brukeropplevelse og gjør feilhåndtering mer brukervennlig.

```
1  {% extends "base.html" %}
2
3  {% block title %}
4  Feil
5  {% endblock %}
6
7  {% block content %}
8
```

1. Denne siden bruker «base.html» så da slipper jeg å skrive «<html>», «<head>», osv.
2. Setter tittlen på siden til «Feil» og det vises i nettleser-tab.
3. «`{% block content %}`» koden starter innhldet som vises inni «base.html»

```

9   <div class="bg-white p-10 rounded-xl shadow text-center max-w-sm w-full">
10  |   <h1 class="text-2xl font-bold text-red-600 mb-4">
11  |       Serverfeil
12  |   </h1>

```

4. Den koden lager en «kort-boks» i midten med hvit bakgrunn, padding(p-10), runde hjørner, skygge, sentrert tekst og maks bredde.
5. Og en overskrift med stor tekst, med røde farge og margin under (mb-4).

```

13  |
14  |   <p class="text-gray-600">
15  |       Klarte ikke hente værddata.
16  |   </p>
17  |

```

6. Dette koden forklarer feilen til brukeren.

```

18  |   <button
19  |       onclick="location.reload()"
20  |       class="mt-6 bg-blue-500 text-white px-4 py-2 rounded hover:bg-blue-600">
21  |       Prøv igjen
22  |   </button>
23 </div>
24
25 {% endblock %}

```

7. En knapp for at siden skal reloades.
8. Og «{% endblock %}» slutter content-blokken.

base.html:

Base-templatene hadde en feil hvor <meta>-taggene lå utenfor <head>-seksjonen. Dette ble rettet for å følge korrekt HTML-struktur. Koden ble også forbedret med bedre organisering og kommentarer for økt lesbarhet.

```

6   <title>
7   |   {% block title %}{% endblock %}
8   </title>

```

1. Dynamisk tittel fra andre templates

```

9      <script src="https://cdn.tailwindcss.com"></script>
10     <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
11     <style>
12         @import url('https://fonts.googleapis.com/css2?family=Inter:wght@100..900&display=swap');
13         body {
14             font-family: 'Inter', sans-serif;
15             background: #f1f5f9;
16         }
17     </style>

```

2. Laster Tailwind CSS som brukes til styling (class= «...»)
3. Laster min egen CSS-fil fra Flask og jeg stiler HTML med en font fra Google.

```

18     </head>
19     <body class="min-h-screen flex items-center justify-center p-4">
20         {% block content %}
21         {% endblock %}
22     </body>
23
24 </html>

```

4. Fra «{% block content %}», «{% endblock %}» kommer innhold fra andre templat.

app.py:

Prosjektet ble forbedret ved å legge til støtte for flere byer gjennom en Dictionary med koordinater. Det ble gjennomført bruk av request for å hente valgt by fra URL, og caching-systemet ble forbedret til å håndtere ulike lokasjoner. I tillegg ble koden refaktorert med egne funksjoner for databaseoperasjoner, noe som gjør løsningen mer strukturert og vedlike holdbar.

Hva jeg har endra:

```

29
30     CACHE_DURATION = datetime.timedelta(minutes=60)
31     CITIES = {
32         "Oslo": (59.91, 10.75),
33         "Bergen": (60.39, 5.32),
34         "Trondheim": (63.43, 10.39),
35         "Stavanger": (58.97, 5.73),
36         "Tromsø": (69.65, 18.96)
37     }
38

```

1. En dictionary med bynavn, koordinater(lat, lon), for å gjøre at appen støtter flere byer.

```

80 def fetch_weather(city):
81     lat, lon = CITIES[city]
82     url = f"https://api.met.no/weatherapi/locationforecast/2.0/compact?lat={lat}&lon={lon}"
83     headers = {
84         "User-Agent": "KubenWeatherApp/1.0 stefanos"
85     }
86     try:
87         response = requests.get(url, headers=headers)
88         response.raise_for_status()
89         data = response.json()
90         timeseries = data["properties"]["timeseries"][0]["data"]
91         temp = timeseries["instant"]["details"]["air_temperature"]
92         symbol = timeseries["next_1_hours"]["summary"]["symbol_code"]
93         description = symbol.replace("_", " ").title()
94         return WeatherData(
95             lokasjon=city,
96             temperatur=temp,
97             beskrivelse=description
98         )
99

```

2. «lat, lon = CITIES[city]», bruker valgt by finner koordinater automatisk.

```

130 def get_weather_from_db():
131     db = get_db()
132     cursor = db.cursor(dictionary=True)
133     cursor.execute("SELECT * FROM vaerdata WHERE id = 1")
134     row = cursor.fetchone()
135     if not row:
136         return None
137
138     return WeatherData(
139         lokasjon=row["lokasjon"],
140         temperatur=row["temperatur"],
141         beskrivelse=row["beskrivelse"],
142         sist_oppdatert=row["sist_oppdatert"]
143     )

```

3. Det henter værdata fra database, men hva er annerledes med det er at før hentet direkte i route, nå det er sitt egen funksjon.

```

144
145 @app.route("/")
146 def index():
147     city = request.args.get("city", "Oslo")
148     weather_db = get_weather_from_db()
149     cache_status = "Ingen data"
150     if weather_db and weather_db.lokasjon == city:
151         age = datetime.datetime.now() - weather_db.sist_oppdatert
152         if age < CACHE_DURATION:
153             minutes = int(age.total_seconds() / 60)
154             cache_status = f"Cache ({minutes} min gammel)"
155             return render_template(
156                 "index.html",
157                 vær=weather_db,
158                 lokasjon=city,
159                 cache_status=cache_status,
160                 cities=CITIES
161             )
162
163     weather_api = fetch_weather(city)
164     if weather_api:
165         update_db(weather_api)
166         cache_status = "Hentet fra Met.no API"
167         return render_template(
168             "index.html",
169             vær=weather_api,
170             lokasjon=city,
171             cache_status=cache_status,
172             cities=CITIES
173         )
174     return render_template("error.html")
175

```

4. "if weather_db and weather_db.lokasjon == city:" jekker også data er for riktig by, så riktig by = riktig data.
5. «cache_status = f"Cache ({minutes} min gammel)» viser hvor gammel data er.
6. «cities=CITIES» sender bylista til templatene og resultatene er dropdown i HTML-funkter.
7. «return render_template("error.html")» er at når det er error brukerens får feilmelding med egen feilside.